

Presentación de Proyecto Final

Ingeniería en Sistemas de Información

Knowgraph: Desarrollo de una Aplicación para
Gestión, Consulta y Visualización de Grafos de
Conocimiento

Dr. Sergio Alejandro Gómez y Valentino Russo

26 de agosto de 2026

Universidad Nacional del Sur

- **1. Motivación y proyectos similares**
- **2. Conocimientos necesarios para utilizar la aplicación**
- **3. Comportamiento de la aplicación y manual de usuario**
- **4. Documentación para desarrolladores**
- **5. Bibliografía**

Motivación y proyectos similares

Motivación

- KnowGraph es una aplicación para operar sobre grafos de conocimiento de forma intuitiva y rápida, con visualización clara y eficiente.
- Facilita la exploración de datos interconectados sin requerir conocimientos profundos de diseño de grafos de conocimiento.
- Integra consultas SPARQL, visualización interactiva y carga de datos heterogéneos.

Proyectos y herramientas relevantes

- **Protégé**
- **MindBugs Discovery**
- **Stardog**
- **Wikidata Query Creator**
- **DBpedia Query Creator**

Protégé

- **Protégé**: editor de ontologías de código abierto (Stanford).



Mindbugs Discovery

- **MindBugs Discovery:** plataforma de análisis semántico y minería de datos para narrativas complejas.



- [illegible]

WikiData Query Creator

- **Wikidata Query Creator:** generación visual de consultas sobre Wikidata.

Wikidata Query Builder

The Wikidata Query Builder provides a visual interface for building a simple Wikidata query. It is ideal for users with little or no experience in [SPARQL](#), the powerful query language. The Query Builder doesn't offer SPARQL's full functionality, but you can always open your query in the Query Service, where you can view, edit or expand it via the link above the results. [Feedback is welcome here](#).

Query

Find all items...

Add condition

Settings

☒ Limit the number of results to

☐ Show IDs instead of labels (may prevent timeout)

[Show query in the Query Service](#)

Results

Results will be displayed here

DBpedia Query Creator

- **DBpedia Query Creator:** acceso a datos estructurados de Wikipedia mediante SPARQL.

Semantic Wikipedia - x

querybuilder.dbpedia.org

The DBpedia query builder is currently redesigned. It will probably be available again after we finish work on the DBpedia live extraction.

The query builder below allows you to build your own queries. Prefix variables with "?".

Subject	Predicate	Object
?actors	rdf:type	dbpedia:Actor

[Update Results](#) [reset](#) [show query](#)

If you think your query can be useful for others, please share:

[Save query](#)

Click on a column header to sort results on this page. Results: 10

actors
db:Alan_Tam
db:Alex_Milison-Jones
db:Brad_Pitton
db:Brian_Banks
db:Chikara_Suzuki
db:Chris_Jones
db:Christian_Vaden
db:George_Lam
db:Robeca_Linares
db:Talissa_Faj

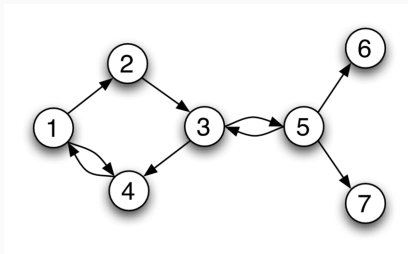
Conocimientos necesarios

Conocimientos necesarios para utilizar la aplicación

- **2.1 Grafos y representación de conocimiento**
- **2.2 Grafos de conocimiento**
- **2.3 RDF y su relación con los grafos**
- **2.4 Ontologías**
- **2.5 SPARQL**
- **2.6 Docker**

Grafos y representación de conocimiento

- Un grafo es una estructura matemática con nodos (entidades) y aristas (relaciones).
- Pueden ser dirigidos o no dirigidos, etiquetados o no.
- Modelan relaciones entre datos de forma natural y flexible.



Grafos de conocimiento

- Combinan grafos con semántica explícita en entidades y relaciones.
- Permiten modelar, integrar y explotar grandes volúmenes de información heterogénea.
- Base para la Web Semántica y los datos enlazados (Linked Data).



RDF y su conexión con los grafos

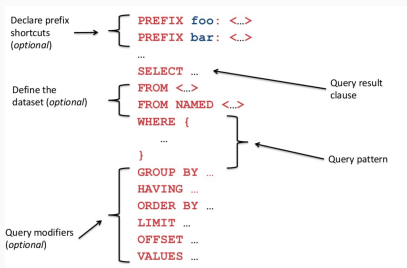
- Resource Description Framework (RDF): estándar W3C para representar información en la Web Semántica.
- Modelo de datos orientado a grafos: sujeto – predicado – objeto (triplas).
- Interoperable, extensible y procesable por máquinas.
- Base de SPARQL y los grafos de conocimiento.

Ontologías

- Especificación formal y explícita de una conceptualización compartida.
- Se componen clases, propiedades, relaciones jerárquicas, restricciones e instancias.
- Lenguajes como RDFS y OWL, añaden semántica formal para el razonamiento automático.
- Facilitan la interoperabilidad semántica, la integración de datos y la inferencia de nuevo conocimiento.

SPARQL

- Lenguaje estándar (W3C) para consultar y actualizar datos RDF.
- Diseñado para grafos semánticos con relaciones complejas.



SPARQL

- Más flexible que SQL para consultas sobre datos enlazados; evita múltiples JOINS.
- Elegido en el proyecto por su expresividad y adecuación a grafos de conocimiento.

Docker

- Plataforma de código abierto para contenerización de aplicaciones.
- Empaqueta la aplicación con todas sus dependencias, garantizando portabilidad.
- Fundamental para que el proyecto se ejecute de manera sencilla en cualquier máquina.
- La primera ejecución puede demorar, pero las subsecuentes son mucho más rápidas.



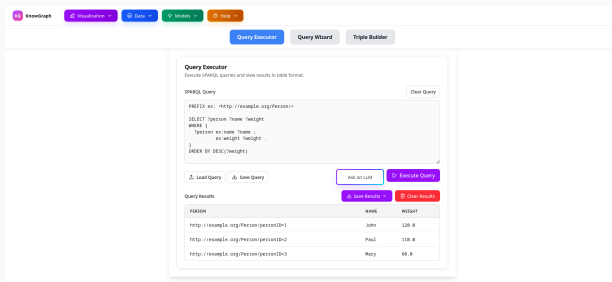
Comportamiento de la aplicación y manual de usuario

Comportamiento de la aplicación y manual de usuario

- **3.1 Ejecución de consultas**
- **3.2 Generador visual de consultas SPARQL**
- **3.3 Constructor de triplas RDF para inserciones**
- **3.4 Cambio de proveedor de servicios SPARQL**
- **3.5 Visualización de datos en KnowGraph**
- **3.6 Ejemplo de uso**

Ejecución de consultas

- Seleccionar *Query Executor*, escribir una consulta SPARQL válida y ejecutar.
- Resultados mostrados en tabla paginada, descargables como CSV o JSON.
- Botón de ayuda que abre un chat con un LLM para resolver dudas sobre SPARQL.



Generador visual de consultas SPARQL

- Se accede mediante el botón *Query Wizard*.
- Bloque superior: generador de prefijos (nombres cortos para URIs).
- Bloque intermedio: manejo de datasets (conjuntos de datos nombrados).

The screenshot displays the KnowGraph application interface. At the top, there is a navigation bar with tabs: 'KnowGraph', 'Visualization', 'Data', 'Metadata', and 'Help'. Below this, a secondary bar contains three buttons: 'Query Executor', 'Query Wizard' (which is highlighted in blue), and 'Triple Builder'. The main content area is divided into three sections:

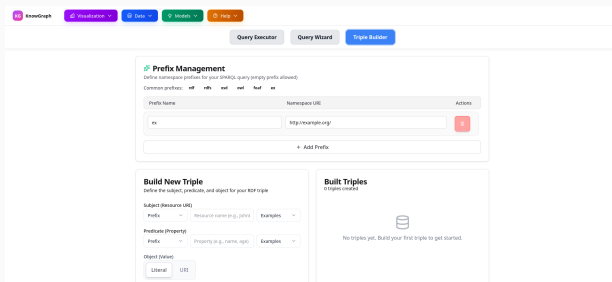
- Prefix Management**: A section for defining namespace prefixes. It includes a table with columns 'Prefix Name' and 'Namespace URI'. The table contains one entry: 'rdf' with the URI 'http://www.w3.org/1999/02/22-rdf-syntax-ns#'. Below the table is an '+ Add Prefix' button.
- Dataset Management**: A section for specifying default and named graphs. It includes a table with columns 'Graph Type' and 'Graph URI'. Below the table is an '+ Add Dataset' button.
- Query Builder**: A section for building SPARQL queries using a visual wizard. It includes a 'Query Description' field.

Generador visual de consultas SPARQL

- Bloque principal: construcción visual de la consulta, seleccionando campos, límite y offset.
- Al generar, la consulta se copia automáticamente al campo de ejecución.
- Permite crear consultas complejas de manera visual.

Constructor de triplas RDF para inserciones

- Similar al generador visual, pero orientado a añadir datos.
- Permite definir sujeto, predicado y objeto (con prefijos o valores literales/URI).

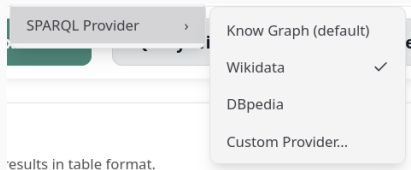


Constructor de triplas RDF para inserciones

- Cada tripla se muestra en un panel lateral; se pueden agregar múltiples.
- Selector de grafo destino (por defecto, el grafo por defecto).
- Al generar, se crea la consulta de inserción y se transfiere al ejecutor.

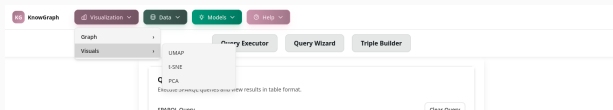
Cambio de proveedor de servicios SPARQL

- Permite cambiar quién responde las consultas SPARQL.
- Proveedor por defecto: servidor SPARQL local (contenedor Docker), que soporta carga de datasets locales/remotos y exportación a múltiples formatos.
- Proveedor externo (ej. Wikidata): consultas sobre sus conjuntos de datos públicos, pero sin carga/exportación local.



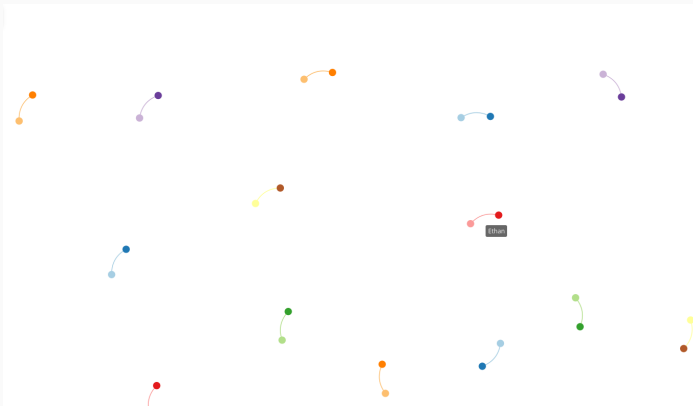
Visualización de datos en KnowGraph

- Se pueden visualizar resultados de consultas de dos formas: grafo o reducción dimensional, cada una con versiones 2D y 3D.
- **Visualización en grafo:** elegir variables que actúan como nodos y arcos, luego definir relaciones. Se puede reutilizar un grafo ya construido.
- **Reducción dimensional** (PCA, t-SNE, UMAP): se seleccionan columnas como características y una columna como etiqueta. Se genera una proyección en 2D o 3D que permite inspeccionar relaciones multivariadas.



Ejemplo de uso en la aplicación

Ahora veamos un ejemplo dentro de la aplicación



Documentación para desarrolladores

Documentación para desarrolladores

- **4.1 Introducción y arquitectura general**
- **4.2 Backend Java**
- **4.3 Backend Python**
- **4.4 Frontend**
- **4.5 Integración y flujos de trabajo**
- **4.6 Docker: contenerización y orquestación**
- **4.7 Nginx como proxy inverso**
- **4.8 Puntos fuertes, débiles y posibles mejoras**

Introducción y arquitectura general

- La aplicación se compone de tres microservicios y un frontend.
- Se utiliza Nginx como proxy inverso para unificar el acceso desde el frontend.
- Docker conteneriza cada servicio, y Docker Compose orquesta el conjunto.
- Comunicación interna mediante red compartida.

Backend Java

- **Apache Jena.**
- **Spring Boot.**
- **Maven.**

Backend Java: Apache Jena

- **Apache Jena:** framework central para manipulación de grafos RDF, ejecución de consultas SPARQL y razonamiento.



Backend Java: Spring Boot

- **Spring Boot:** framework web que expone los endpoints de comunicación con el frontend.



Backend Java: Maven

- **Maven:** herramienta de construcción y gestión de dependencias que empaqueta todo en un ejecutable de producción.

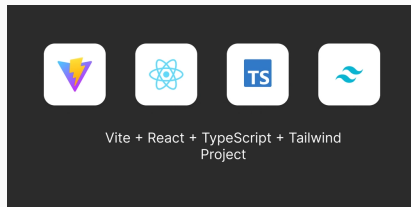


Backend Python

- **Importer:** servicio Flask que convierte archivos estructurados (CSV, JSON, Excel, XML) en grafos RDF, mediante detección de formato y generación de triplas.
- **Visualizer:** servicio Flask que realiza reducción dimensional (PCA, t-SNE, UMAP) sobre datos tabulares, generando proyecciones 2D/3D listas para visualizar.
- Ambos utilizan Gunicorn como servidor WSGI y se comunican a través de la red interna de Docker.

Frontend

- Construido con el framework de desarrollo web Vite con React y el lenguaje Typescript y tailwindcss para darle estilos a la pagina.
- Se usa React Router para acceder a las distintas paginas.



Frontend

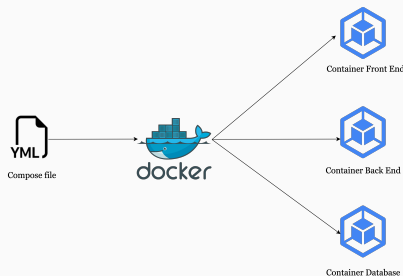
- Se comunica con los microservicios a través de las rutas proxy definidas en Nginx.
- Proporciona la interfaz de usuario para ejecutar consultas, construir visualizaciones y gestionar datos.

Integración y flujos de trabajo

- **Comunicación entre componentes:** el frontend envía peticiones HTTP a Nginx, que las redirige al servicio correspondiente según la ruta.
- Los servicios Python y Java se invocan mediante API REST.
- Docker Compose garantiza el despliegue coordinado y la red común.

Docker: contenerización y orquestación

- Cada servicio (sparkl-backend, importer, visualizer, frontend) tiene su propio contenedor.
- Docker Compose define la configuración, redes, volúmenes y variables de entorno.
- Facilita la portabilidad y el despliegue en cualquier entorno compatible con Docker.



Nginx como proxy inverso

- Unifica el acceso desde el puerto 80 hacia los distintos servicios.
- Configuración de `nginx.conf` con reglas de proxy para las rutas `/api/sparql`, `/api/importer`, `/api/visualizer`.



Nginx

- Sirve los estáticos del frontend y redirige las peticiones no encontradas al `index.html` (SPA).
- Timeouts amplios para consultas pesadas, por ejemplo a la hora de generar los puntos para reducción dimensional y visualizarlos.

Puntos fuertes, débiles y posibles mejoras: Puntos fuertes

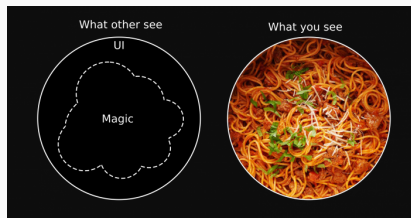
- Fácil despliegue en cualquier dispositivo que soporte Docker.
- Amplia funcionalidad sobre **datasets locales**.
- Visualización interactiva de resultados tanto en 2d como en 3d.
- Asistente integrado basado en LLM para resolver dudas sobre consultas SPARQL.

Puntos fuertes, débiles y posibles mejoras: Puntos débiles

- La instalación de Docker puede resultar compleja para un usuario sin perfil técnico y la primera ejecución tarda mucho por la descarga de imágenes.
- El servicio **Visualizer** utiliza una imagen base pesada lo que incrementa el tamaño final del contenedor.
- Al cambiar a un proveedor de datos externo (ej. Wikidata), múltiples funcionalidades dejan de estar disponibles.
- Deuda técnica en el código.

Puntos fuertes, débiles y posibles mejoras: Posibles mejoras

- Refactorización del código para mejorar mantenibilidad.
- Extender el servicio de *Importer* con soporte para más tipos de archivo y fuentes remotas (URLs, APIs).
- Añadir nuevos métodos de visualización.
- Incorporar una guía de ayuda interactiva dentro de la propia aplicación.
- Implementar una barra de búsqueda y filtrado para localizar datos específicos en los resultados de una consulta.



Bibliografía

Referencias bibliográficas (1)

-  Bob Ducharme, *Learning SPARQL* 2nd edition, O'Reilly Media, 2013.
-  T. Berners-Lee, *Linked Data*, W3C, 2006.
-  A. Hogan et al., *Knowledge Graphs*, 2020.
-  W3C, *Resource Description Framework (RDF)*, W3C Recommendation, 2014.
-  W3C, *SPARQL 1.1 Query Language*, W3C Recommendation, 2013.
-  Apache Jena, *Documentation*,
<https://jena.apache.org/documentation/>.
-  Spring Boot Documentation,
<https://spring.io/projects/spring-boot>.

Referencias bibliográficas (2)



Docker Inc., *Docker Documentation*,
<https://docs.docker.com/>.



Vite, *Guía de inicio*, <https://vitejs.dev/>.



NGINX, *Documentación*, <https://nginx.org/en/docs/>.

Gracias por su atención

¿Preguntas?

Contacto: valentinori870@gmail.com